Initialization, I/O, Analysis & Parallelization
of Polyethylene Chains

Itxaso Muñoz-Aldalur (itxasoma)

Contents

1 Initial Configuration Generation

1.1 Internal-Coordinate Builder

The `initial_conf` module is responsible for generating the starting conformation of the polyethylene chain. The builder uses a recursive internal-coordinate approach: each successive carbon atom is placed by specifying a fixed C–C bond length ($d = 1.54\,\text{Å}$), a fixed C–C–C bond angle ($\theta = 114°$), and a selectable dihedral angle ϕ. A hard-sphere overlap check with threshold $r_{\min} = 2.0\,\text{Å}$ is performed after each placement to avoid unphysical self-intersections during chain construction.

```
! Place carbon i based on the last two accepted positions
do i = 4, n_carbons
  ! Bond vector from i-2 to i-1 (normalised)
  b1 = coords(i-1,:) - coords(i-2,:)
  b1 = b1 / vnorm(b1)

  ! Normal to the C(i-2)-C(i-1)-C(i) plane
  b0 = coords(i-2,:) - coords(i-3,:)
  n_vec = cross(b0, b1)
  if (vnorm(n_vec) > 1.0d-8) n_vec = n_vec / vnorm(n_vec)

  ! Proposed new position using dihedral phi
  d_vec = bond_length * (                              &
      -cos_theta * b1                                  &
      + sin_theta * (cos_phi * n_vec                   &
                   + sin_phi * cross(b1, n_vec)))

  candidate = coords(i-1,:) + d_vec

  ! Overlap check against all previously placed atoms
  overlap = .false.
  do j = 1, i-1
    if (vnorm(candidate - coords(j,:)) < r_min) then
      overlap = .true.
      exit
    end if
  end do
  if (.not. overlap) coords(i,:) = candidate
end do
```

Listing 1: Core internal-coordinate placement loop in `initial_conf.f90`.

When explicit hydrogens are requested (`explicit_h = .true.`), two hydrogen atoms are appended for each internal CH_2 group and one for each terminal CH_3 group, giving $N_{\text{atoms}} = 3N_C + 2$ total atoms for a chain of N_C carbons. The hydrogen placement is based on a local tetrahedral frame: instead of placing the H atoms coplanar with the C–C–C backbone (chemically inconsistent), they are positioned out of the backbone plane at the correct tetrahedral angle of $\approx 109.5°$.

```
! For each internal carbon i (not first or last):
! Build local orthonormal frame {b_fwd, perp, out_of_plane}
b_fwd = coords(i+1,:) - coords(i-1,:)
b_fwd = b_fwd / vnorm(b_fwd)

! Perpendicular to backbone, in the C(i-1)-C(i)-C(i+1) plane
b_back = coords(i-1,:) - coords(i,:)
b_back = b_back / vnorm(b_back)
```

```
perp   = b_back - sum(b_back*b_fwd)*b_fwd
if (vnorm(perp) > 1.0d-8) perp = perp / vnorm(perp)

! Out-of-plane direction (cross product)
out_of_plane = cross(b_fwd, perp)
out_of_plane = out_of_plane / vnorm(out_of_plane)

! Place H1 and H2 symmetrically out of plane (tetrahedral geometry)
! cos(109.5 deg) ~ -1/3   =>  sin(109.5 deg) ~ sqrt(8/9)
h1 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp)  &
     + sqrt(8.0d0/9.0d0) * 0.5d0 * ( out_of_plane + b_fwd) )
h2 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp)  &
     + sqrt(8.0d0/9.0d0) * 0.5d0 * (-out_of_plane + b_fwd) )
```

Listing 2: Out-of-plane hydrogen placement for internal CH_2 groups.

1.2 Configuration Modes

Five distinct initial configuration modes have been implemented, each designed to probe a different region of the polymer's conformational space:

Table 1: Summary of implemented initial configuration types.

conf_type	Name	Description
1	All-trans (planar)	All dihedrals set to $\phi = 0°$; fully extended zigzag chain. Maximum end-to-end distance.
2	Random	Each dihedral drawn uniformly from $[-\pi, \pi]$; maximally disordered starting state.
3	RIS-like discrete	Dihedrals assigned with probabilities reflecting the Rotational Isomeric State model (trans, gauche$^+$, gauche$^-$).
4	Spring / helix	Progressive dihedral increment per bond, producing a helical backbone (proposed by ai-murphy).
5	Perturbed trans	Small random perturbation applied to the all-trans geometry; slight out-of-plane deviation while remaining close to the global minimum.

Configurations 1 and 5 span the near-trans region of phase space, whereas configurations 2 and 3 provide disordered starting points. Configuration 4 was added to test helical initial geometries. This variety is relevant when the simulation is tested at different temperatures: some initial states may lie closer to the equilibrated ensemble than others, reducing the effective burn-in time.

2 Input/Output Module

2.1 Parameter Parsing from `input.dat`

All simulation parameters are read from a plain-text file `input.dat` through the `io` module. The parser applies default values for any missing keyword, strips inline comments (lines beginning with `#`), and performs basic range validation. The following parameters are controlled from the input file:

Output is written in XYZ format: trajectories to `traj_`*`label`*`.xyz`, energies to `energy_`*`label`*`.dat`,

Table 2: Simulation parameters read from `input.dat`.

Parameter	Default	Description
`n_carbons`	10	Number of backbone carbon atoms
`n_steps`	1,000,000	Number of MC production steps
`temperature`	300.0	Simulation temperature (K)
`conf_type`	1	Initial configuration mode (see Table 1)
`explicit_h`	.false.	Enable all-atom (explicit hydrogen) mode
`rng_seed`	42	Random number generator seed
`print_interval`	1,000	Steps between output writes
`max_delta`	0.5	Maximum dihedral rotation per MC step (rad)

and observables to `observables_`*`label`*`.dat`, where the label encodes the key simulation parameters (`n_carbons`, `conf_type`, `n_steps`, `temperature`) for unambiguous identification.

3 Serial Driver and Scientific Output

3.1 Two-Phase Simulation Structure

The serial driver `main_serial_equil.f90` is organised into two sequential phases. Phase 1 (equilibration) runs until the Geweke convergence diagnostic (see the Monte Carlo Engine section) declares stationarity and phase 2 (production) that restarts the step counter and accumulates the data used for analysis.

3.2 Annealing Schedule

An optional simulated annealing schedule is implemented in Phase 1 to help the chain escape local energy minima. The temperature decreases linearly from T_{ini} to T_{fin} over the first `n_steps` equilibration steps:

$$T(t) = T_{\text{ini}} - \Delta T \cdot (t-1), \qquad \Delta T = \frac{T_{\text{ini}} - T_{\text{fin}}}{n_{\text{steps}} - 1} \tag{1}$$

When `T_ini = T_fin`, the schedule reduces to a constant-temperature run (the default production setting). The annealing feature was used during development to verify that the code produces physically consistent results: as $T \to 0\,\text{K}$, the polymer is expected to relax towards the fully extended all-trans conformation (`conf_type = 1`), which corresponds to the global energy minimum for the chosen force-field parameters. This behaviour was confirmed in all test runs where the chain did not become kinetically trapped in twisted or tangled states.

3.3 CPU Timing and Performance Reference

The intrinsic `cpu_time` routine has been used to measure the equilibration and production phases independently, separating the associated computational costs. These timings are written as output files, giving a baseline for the serial performance baseline later compared with the parallel implementations.

4 Python Post-Processing and Analysis

4.1 Automated Equilibration Detection

Before any observable is plotted, the Python script `plot_results.py` applies an automated equilibration detector to the energy time series. The method follows Chodera's paper [**?**], which selects the discard index t_0 that maximises the number of effectively independent samples N_{eff}:

$$N_{\text{eff}}(t_0) = \frac{N - t_0}{g(t_0)}, \qquad g = 1 + 2\,\tau_{\text{int}} \tag{2}$$

where g is the statistical inefficiency and τ_{int} is the integrated autocorrelation time, computed via Fast Fourier Transform (FFT):

```
def detect_equilibration(x):
    # Chodera (2015): find t0 that maximises N_eff
    n = len(x)
    best_neff, best_t0 = 0.0, 0
    candidates = np.unique(
        np.linspace(0, int(n * 0.9), min(200, n)).astype(int))
    for t0 in candidates:
        sub = x[t0:]
        _, g = integrated_autocorr_time(sub)
        neff = (n - t0) / g
        if neff > best_neff:
            best_neff, best_t0 = neff, t0
    return best_t0

def integrated_autocorr_time(x):
    # FFT-based normalised autocorrelation function
    n   = len(x)
    xc  = x - np.mean(x)
    f   = np.fft.rfft(xc, n=2*n)
    acf = np.fft.irfft(f * np.conj(f))[:n]
    acf /= acf[0]
    # Truncate at first negative value (Sokal window)
    cut = next((i for i in range(1, n) if acf[i] < 0.0), n)
    tau_int = 0.5 + np.sum(acf[1:cut])
    return tau_int, 1.0 + 2.0 * tau_int   # tau_int, g
```

Listing 3: Equilibration detector and autocorrelation-time estimator.

This approach is better than a fixed burn-in fraction because it adapts to the actual autocorrelation structure of each run, maximising statistical efficiency and not discarding more data than necessary. Then, Torsion angle histograms are constructed exclusively from data collected after t_0. The empirical distribution is overlaid with the theoretical Boltzmann weight derived from the torsional potential (TraPPE-UA or OPLS-AA effective backbone).

5 Cluster Compatibility

Portability to shared HPC infrastructure required two targeted modifications. First, the `Makefile` replaces the Fortran 2008 `open(newunit=u)` syntax with explicit unit-number declarations compatible with Fortran 90 compilers available on the CERQT2 cluster. Second, a submission script `1.run.sh` was created to load the appropriate GCC and OpenMPI modules before compilation and execution. The `parameters.f90` file was adapted to restore the `#ifdef` preprocessor block

controlling the `explicit_h` compilation flag, and a comment was added to the `Makefile` explaining why the `-I$(MPI_INC_DIR)` flag is conditionally omitted in certain build configurations.

6 MPI Parallelization: Independent Replicas

6.1 Justification

The MPI parallelization implemented here is based on independent replica parallelism as a first step, while the energy evaluation and the Monte Carlo kernel can be parallelized subsequently (see the Energy Evaluation and Monte Carlo sections). First, independent replicas do not require inter-process communication inside the Monte Carlo loop, which keeps synchronization overhead very low. Second, running several initial configurations simultaneously makes it possible to assess how sensitive the final sampled ensemble is to the choice of starting geometry.

6.2 Implementation: `main_parallel_replicas.f90`

Starting from `main_serial.f90`, three MPI ranks are launched simultaneously. Each rank runs the identical Monte Carlo protocol (same thermodynamic parameters, same number of steps) but is assigned a distinct initial configuration and a perturbed random seed:

```
! MPI initialization
call MPI_Init(ierr)
call MPI_Comm_rank(MPI_COMM_WORLD, rank, ierr)
call MPI_Comm_size(MPI_COMM_WORLD, n_ranks, ierr)

! Assign initial configuration based on rank
select case (rank)
  case (0); conf_type = 1   ! all-trans (extended)
  case (1); conf_type = 4   ! helix / spring
  case (2); conf_type = 5   ! perturbed trans
end select

! Perturb the RNG seed so trajectories immediately diverge
rng_seed = rng_seed + rank * 104729

! Build the initial configuration and run the full MC protocol
call build_initial_conf(n_carbons, conf_type, explicit_h, coords,
   symbols)
call mc_run(...)

! Rank-labelled output so files do not overwrite each other
write(label, '(A,I1)') trim(base_label)//'_rank', rank
call write_trajectory(label, coords, symbols, n_atoms)
```

Listing 4: Rank-dependent configuration and seed assignment.

The three configurations chosen (`conf_type` 1, 4, and 5) cover the near-trans region from three distinct but close starting points: fully extended, helical, and slightly perturbed extended. This choice maximises the geometric diversity of the initial ensemble while keeping all replicas in a physically reasonable region of conformational space.

A serial counterpart, `main_serial_replicas.f90`, runs the same three configurations sequentially to provide a controlled performance reference. Both versions employ wall-clock timing: `MPI_Wtime` in the parallel driver and an equivalent system-clock call in the serial one, ensuring that the speedup ratios reported in Section 8 reflect actual elapsed time rather than CPU time.

6.3 Discarded Parallelization: XYZ Trajectory Writing

A second parallelization direction was explored: distributing the XYZ trajectory write step across ranks, so that each rank writes its own frame independently. The resulting wall-clock improvement was marginal (less than 2% of total runtime), confirming that trajectory I/O is not the computational bottleneck. The dominant cost lies in the energy evaluation and Metropolis acceptance/rejection logic, as expected from the $\mathcal{O}(N^2)$ Lennard-Jones loop. This branch was therefore discarded and the simpler rank-labelled sequential write was kept.

7 MPI Parallelization: Observable Post-Processing

7.1 Design and Implementation

Post-processing of the trajectory data has been parallelized in `main_parallel_observables.f90`. Rather than decomposing a single trajectory into contiguous frame blocks, the implementation operates at the file level: all trajectory files are first enumerated in a manifest, rank 0 reads this manifest, and the complete file list is broadcast to all MPI ranks. Each rank then processes a round-robin subset of trajectory files, independently computing the radius of gyration R_g, the end-to-end distance R_{ee}, and the torsional statistics for the configurations assigned to it.

This design was chosen because the available data are naturally distributed across multiple trajectory files, corresponding to different configuration types and simulation phases (`equil` and `prod`). A file-level distribution avoids repeated file seeks inside large XYZ trajectories, keeps the implementation simple, and preserves a fully embarrassingly parallel local workload. At the same time, it remains fully compatible with collective MPI communication for global aggregation.

A first collective step is performed through `MPI_Bcast`, which is used to distribute the manifest and the maximum number of torsional degrees of freedom to all ranks. After each rank has completed its local file subset, a mid-run checkpoint is evaluated by means of `MPI_Allreduce`. In this step, the partial frame counts and partial R_g sums are synchronised across all processes, so that the global partial mean can be computed before the final aggregation stage. This checkpoint was introduced as a lightweight example of active communication during the computation phase, beyond the final collection of results.

```
! Each rank processes a round-robin subset of trajectory files
do ifile = rank + 1, nfiles, num_procs
  call process_trajectory(trim(files(ifile)), max_phi, &
       local_file_count, local_frame_count, &
       local_sum_rg, local_sum_rg2, local_sum_ree, local_sum_ree2, &
       local_nphi, local_sum_phi, local_sum_phi2)
enddo

! Mid-run checkpoint: all ranks obtain the global partial mean of Rg
call MPI_Allreduce(local_frame_count, ckpt_frame_count, n_conf*n_phase
   , &
                   MPI_INTEGER, MPI_SUM, MPI_COMM_WORLD, ierr)
call MPI_Allreduce(local_sum_rg, ckpt_sum_rg, n_conf*n_phase, &
                   MPI_DOUBLE_PRECISION, MPI_SUM, MPI_COMM_WORLD, ierr
                      )

! Final aggregation on rank 0
call MPI_Reduce(local_sum_rg, global_sum_rg, n_conf*n_phase, &
                MPI_DOUBLE_PRECISION, MPI_SUM, 0, MPI_COMM_WORLD, ierr
                   )
```

Listing 5: Round-robin file distribution and MPI checkpoint in

`main_parallel_observables.f90`.

The final global statistics are obtained on rank 0 through a series of `MPI_Reduce` operations, which combine the local sums, squared sums, frame counts, and torsional accumulators into global averages and standard deviations. The corresponding summary files (`summary_global_np1.dat`, `summary_global_np2.dat`, `summary_global_np4.dat`, and `summary_global_np8.dat`) show that the numerical results are identical for all tested values of n_p, confirming that the parallelization preserves the observable averages exactly.

The benchmark driver is invoked from the cluster submission script `3.run_parallel_observables_np.sh`, which executes the same post-processing workflow for $n_p \in \{1, 2, 4, 8\}$, records both MPI and shell wall-clock times, and writes per-process summary files for subsequent comparison.

8 Scaling Analysis

8.1 Replica Parallelism: Speedup vs. System Size and Run Length

The independent-replica driver (`main_parallel_replicas.f90`) was benchmarked against its serial counterpart (`main_serial_replicas.f90`) across a range of chain lengths ($N_C \in \{20, 50, 100, 200, 500\}$) at fixed $n_{\text{steps}} = 10^6$, and across a range of run lengths ($n_{\text{steps}} \in \{10^5, 10^6, 10^7\}$) at fixed $N_C = 100$. All runs used $N_{\text{ranks}} = 3$ MPI processes.

Table 3: Replica speedup as a function of chain length ($n_{\text{steps}} = 10^6$, 3 MPI ranks vs. 3 sequential serial runs).

N_C	N_{atoms}	n_{ranks}	t_{serial} (s)	t_{parallel} (s)	Speedup
20	42	3	42.71	15.22	2.81
50	102	3	239.32	88.33	2.71
100	202	3	790.35	276.21	2.86
200	402	3	3,418.03	984.09	3.47
500	1002	3	17,755.64	6,113.38	2.90

Table 4: Replica speedup as a function of run length ($N_C = 100$, 3 MPI ranks vs. 3 sequential serial runs).

n_{steps}	n_{ranks}	$n_{\text{serial runs}}$	t_{serial} (s)	t_{parallel} (s)	Speedup
10^5	3	3	76.29	26.11	2.92
10^6	3	3	790.35	276.21	2.86
10^7	3	3	8,112.94	2,883.55	2.81

The speedup remains consistently close to the ideal value of 3 across the majority of chain lengths and run lengths. Since the replicas are fully independent, no communication is required inside the Monte Carlo loop itself, and the only parallel overhead originates from MPI initialization, process launch, and final output management. The slightly sub-linear values ($S \approx 2.8$–2.9) are therefore interpreted as the expected effect of this fixed overhead. The isolated value $S = 3.47$ at $N_C = 200$ is best understood as a small timing fluctuation of the serial reference rather than as a systematic superlinear scaling effect.

8.2 Observable Post-Processing: Strong Scaling

The observable post-processing driver was benchmarked on a fixed dataset for $n_p \in \{1, 2, 4, 8\}$. The resulting MPI wall times are 19.35 s, 9.33 s, 4.97 s, and 3.06 s, respectively. From these

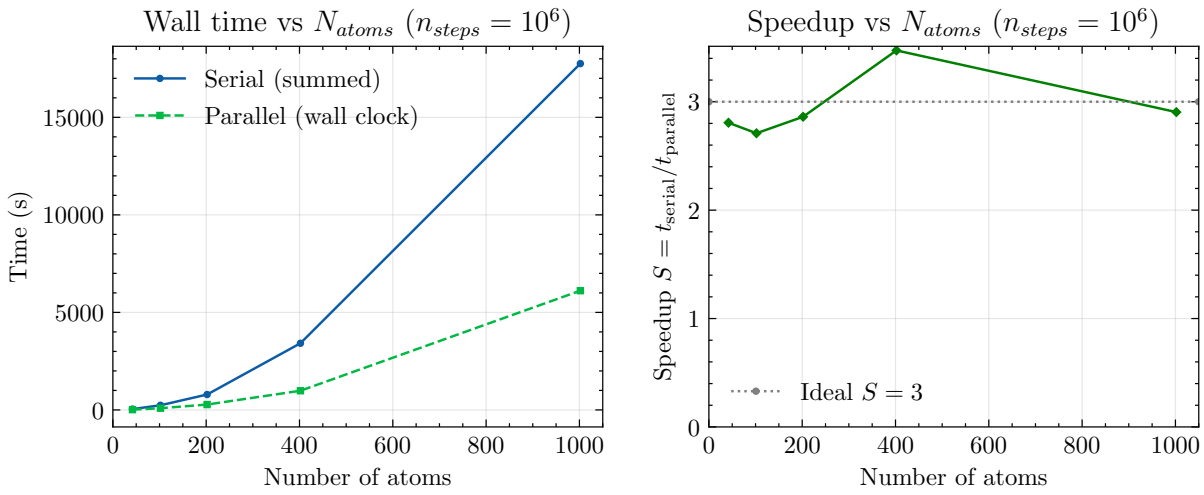

Figure 1: Speedup of independent-replica parallelization: speedup as a function of chain length ($N_C \in \{20, 50, 100, 200, 500\}$, $n_{\mathrm{steps}} = 10^6$). All runs use 3 MPI ranks. The dashed line indicates ideal linear scaling ($S = 3$).

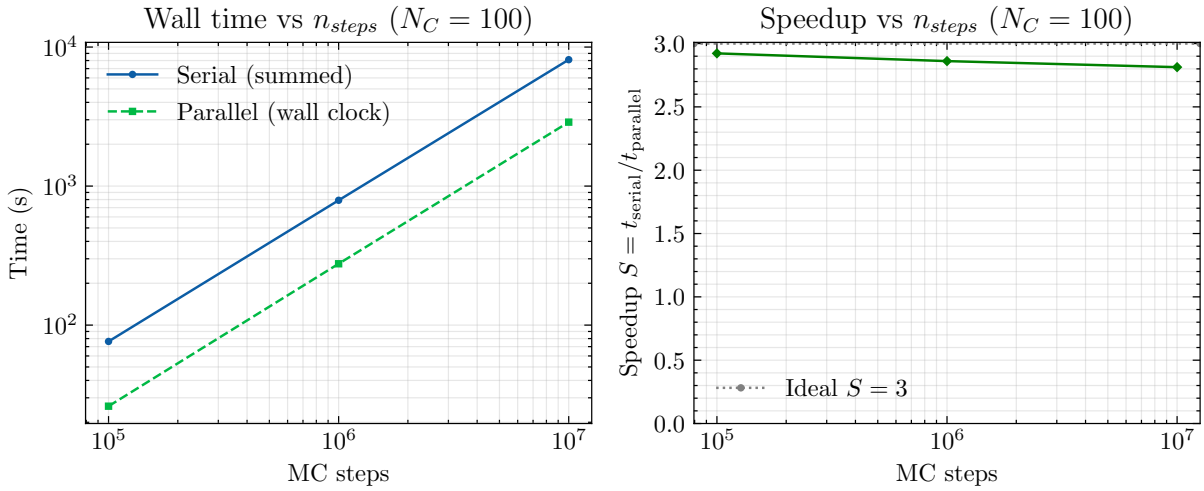

Figure 2: Speedup of independent-replica parallelization: speedup as a function of run length ($n_{\mathrm{steps}} \in \{10^5, 10^6, 10^7\}$, $N_C = 100$). All runs use 3 MPI ranks. The dashed line indicates ideal linear scaling ($S = 3$).

values, the measured speedups are 1.000, 2.074, 3.894, and 6.318, corresponding to efficiencies of 1.000, 1.037, 0.974, and 0.790.

A near-ideal strong-scaling regime is observed up to $n_p = 4$. The slight superlinear efficiency at $n_p = 2$ ($E = 1.037$) is attributed to normal timing variability and low-level runtime effects in the serial baseline, rather than to a genuine algorithmic gain. For this reason, the result is interpreted as essentially ideal scaling within the uncertainty of the benchmark. At $n_p = 8$, the efficiency decreases to 0.790, indicating the expected onset of diminishing returns once the local workload per rank becomes too small and the relative cost of collective communication, process startup, and file I/O becomes non-negligible.

The correctness of the parallel implementation is further supported by the fact that all per-n_p summary files produce identical values of R_g and R_{ee} for every configuration and phase. The MPI decomposition therefore changes the execution time, but not the final numerical averages.

Table 5: Strong scaling of `main_parallel_observables.f90`.

n_p	t_{MPI} (s)	t_{shell} (s)	Speedup	Efficiency
1	19.35	19.70	1.000	1.000
2	9.33	9.53	2.074	1.037
4	4.97	5.31	3.894	0.974
8	3.06	3.29	6.318	0.790

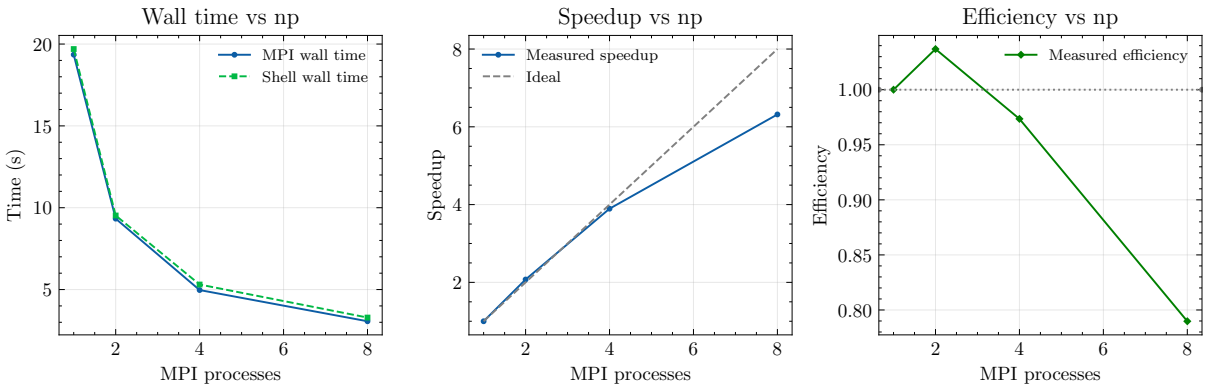

Figure 3: Strong scaling of the observable post-processing kernel. MPI wall-clock time (solid circles) and shell-measured elapsed time (dashed squares) as a function of the number of MPI processes n_p.

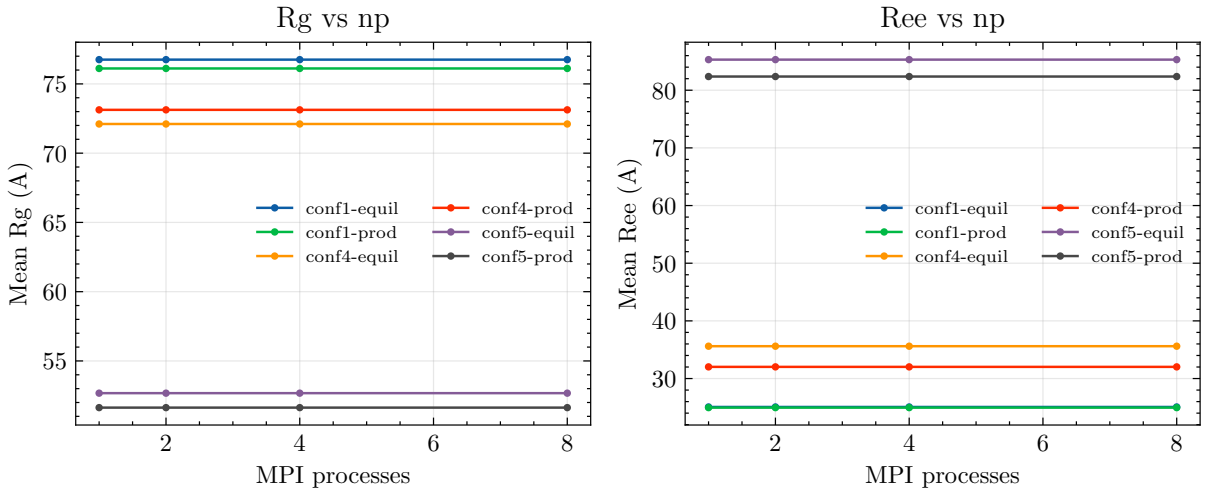

Figure 4: Mean radius of gyration R_g and mean end-to-end distance R_{ee} as a function of the number of MPI processes n_p. The overlap of all curves confirms that the observable averages are preserved exactly across the tested parallel decompositions.

8.3 Discussion

Both parallelization strategies achieve their intended objectives within their respective scopes. The independent-replica implementation provides a clean MPI layer for running several trajectories simultaneously, while the observable post-processing kernel demonstrates that the analysis stage can also be distributed efficiently across ranks.

For the post-processing code, the benchmark indicates near-ideal scaling up to four MPI processes, with only a moderate efficiency loss at eight processes. This behaviour is fully consistent

with the structure of the algorithm: local trajectory analysis is embarrassingly parallel, whereas the final collective steps (`MPI_Allreduce`, `MPI_Barrier`, and `MPI_Reduce`) introduce a fixed communication cost that becomes visible only when the amount of work per rank is sufficiently small.

Taken together, these results establish that the MPI layer developed for the project is not limited to trajectory generation alone, but also extends naturally to post-processing and benchmarking. The observable kernel therefore provides a useful and technically distinct contribution to the overall parallel workflow, complementing the independent-replica and collaborative-equilibration strategies implemented in the simulation stage.

References

- Chodera, J.D. (2016). A simple method for automated equilibration detection in molecular simulations. *J. Chem. Theory Comput.*, 12(4), 1799–1805. 10.1021/acs.jctc.5b00784